

Advanced CSS

Where your styles live, how the box model shapes every element, and laying out real pages with Flexbox and Grid.

WHAT WE COVER

[Inline / External CSS](#)[Classes](#)[Box model](#)[Flexbox](#)[Grid](#)

You can already write HTML and set a few colours. Today we turn that into real design skill: choosing where CSS lives, styling with reusable classes, and — the heart of it — controlling space and layout. By the end you'll build a styled component from scratch.

Advanced CSS

CSS has two jobs: decide *which* elements to style, and decide *how* they look and sit on the page. Master the box model and the two layout systems below and you can build almost any interface you can picture.

ON THIS HANDOUT

1. Where CSS lives: inline, internal & external
2. Selecting things: classes & the essentials
3. Styling basics: colours & text
4. The box model — padding, border & margin
5. Flexbox — one-dimensional layout
6. Grid — two-dimensional layout
7. Putting it together: a styled component
8. Recap & best practices

01 Where CSS lives: inline, internal & external

There are three places you can write CSS. They do the same thing — set styles — but they differ in scope and how maintainable they are.

1. Inline — on a single element

CSS written in a `style` attribute, affecting only that one tag.

```
<p style="color: white; background: #4f46e5;">Hello</p>
```

HTML

Quick for a one-off test, but it clutters your HTML, can't be reused, and is hard to override. Avoid it for real work.

2. Internal — in a `<style>` block

CSS placed inside `<style>` in the page's `<head>`. It styles that *one page*.

```
<head>
  <style>
    p { color: white; background: #4f46e5; }
  </style>
</head>
```

HTML

3. External — a separate `.css` file

CSS in its own file, linked from the HTML. One file can style your *entire site*.

```
<!-- index.html -->
<link rel="stylesheet" href="styles.css">
```

HTML

```
/* styles.css */
p { color: white; background: #4f46e5; }
```

CSS

Method	Scope	Use it when...
Inline	One element	testing a quick idea (rarely).
Internal	One page	a tiny single-page demo.
External	Whole site	always, for real projects.

KEY IDEA

Separation of concerns: keep structure in HTML and style in an external CSS file. Your HTML stays clean, and one edit restyles every page at once.

02 Selecting things: classes & the essentials

Before you can style an element you must *select* it. The most important selector in day-to-day CSS is the **class** — a reusable label you attach to any number of elements.

```
<button class="btn">Save</button>
<button class="btn">Cancel</button>
```

HTML

```
.btn {
  background: #4f46e5;
  color: white;
  padding: 10px 18px;
  border-radius: 6px;
}
```

CSS

A dot (`.`) in CSS means “class.” The rule above styles *both* buttons because they share `class="btn"`. Change the class once and every button updates.

Class vs. id

Selector	Written as	Reuse
Class	<code>.btn</code> → <code>class="btn"</code>	Many elements — your default choice.
id	<code>#nav</code> → <code>id="nav"</code>	One unique element only.

Other selectors you'll use constantly

Selector	Targets
<code>p</code>	Every <code><p></code> by tag name.
<code>.card .title</code>	A <code>.title</code> inside a <code>.card</code> (descendant).
<code>h1, h2, h3</code>	Several selectors at once (grouping).
<code>a:hover</code>	A link while the mouse is over it.

NOTE

Specificity: if two rules target the same element, `#id` wins over `.class`, which wins over a plain `tag`. Prefer classes so your styles stay easy to override.

03 Styling basics: colours & text

With an element selected, you set its look with **property: value** pairs. The two you'll use most are colour and text.

Colours

A colour can be written several ways — all describe the same thing:

```
color: crimson;           /* named */
color: #4f46e5;           /* hex */
color: rgb(79, 70, 229);  /* r,g,b */
color: rgba(0,0,0,0.5);   /* + alpha */

background-color: #f1f5f9; /* the element's fill */
```

CSS

     `color` sets the text; `background-color` sets the fill behind it.

Text

Property	Does
<code>font-family</code>	The typeface: <code>Arial, sans-serif</code> (list fallbacks).
<code>font-size</code>	Size of the text: <code>16px</code> , <code>1.25rem</code> .
<code>font-weight</code>	Thickness: <code>bold</code> or <code>100–900</code> .
<code>text-align</code>	<code>left</code> , <code>center</code> , <code>right</code> .
<code>line-height</code>	Space between lines: <code>1.5</code> reads well.

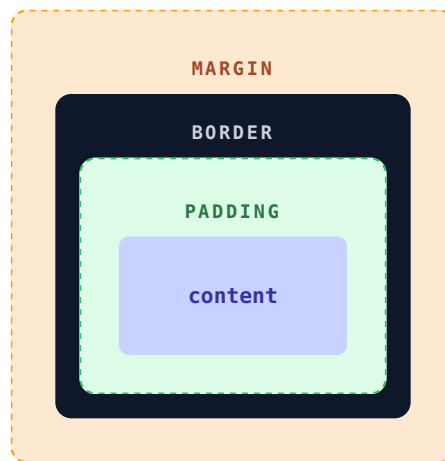
WORKED EXAMPLE — A HEADING STYLE

```
.title {  
  font-family: Arial, sans-serif;  
  font-size: 24px;  
  font-weight: 700;  
  color: #0f172a;  
  text-align: center;  
}
```

CSS

04 The box model — padding, border & margin

This is the idea everything in CSS layout rests on: **every element is a rectangular box**, made of four layers wrapped around your content.



From the inside out: content → padding → border → margin

Layer	What it is
content	The text or image itself — sized by <code>width</code> / <code>height</code> .
padding	Space <i>inside</i> the box, between content and border. Pushes the border out.
border	A line around the padding: <code>2px solid #4f46e5</code> .
margin	Space <i>outside</i> the box, holding other elements away.

KEY IDEA

Easy to mix up: **padding** grows the box from the inside (more room around the content); **margin** pushes other boxes away. Padding is inside the border, margin is outside it.

Writing the values

```
padding: 16px;           /* all four sides */
padding: 10px 20px;      /* top-bottom left-right */
margin: 8px 16px 8px 16px; /* top right bottom left */

border: 2px solid #4f46e5; /* width style colour */
border-radius: 8px;        /* rounded corners */
```

CSS

Shorthand values run **clockwise from the top**: top, right, bottom, left.

WATCH OUT

The box-sizing fix. By default, `width` sets only the content, so padding and border make the box *bigger* than you asked. Add `box-sizing: border-box` and the width includes padding and border — far more predictable. Most projects set it on everything:

```
* { box-sizing: border-box; }
```

05 Flexbox — one-dimensional layout

Flexbox arranges items in a single direction — a row or a column — and shares space between them. It is the go-to tool for navbars, button groups, and centring.

Set `display: flex` on a **container**; its direct children become flexible **items**.

```
.nav {
  display: flex;
  justify-content: space-between; /* spread along the row */
  align-items: center;           /* line up vertically */
  gap: 16px;
}
```

CSS

Logo

Home About Contact

`justify-content: space-between` pushes the logo and links to opposite ends.

The container properties

Property	Controls
<code>flex-direction</code>	<code>row</code> (default) or <code>column</code> .
<code>justify-content</code>	Spacing along the main axis: <code>center</code> , <code>space-between</code> , <code>space-around</code> .
<code>align-items</code>	Alignment across the main axis: <code>center</code> , <code>flex-start</code> , <code>stretch</code> .
<code>flex-wrap</code>	<code>wrap</code> lets items drop to the next line when space runs out.

Property	Controls
<code>gap</code>	Even spacing between items — cleaner than margins.

TIP

Centre anything — the classic trick: on the container, set

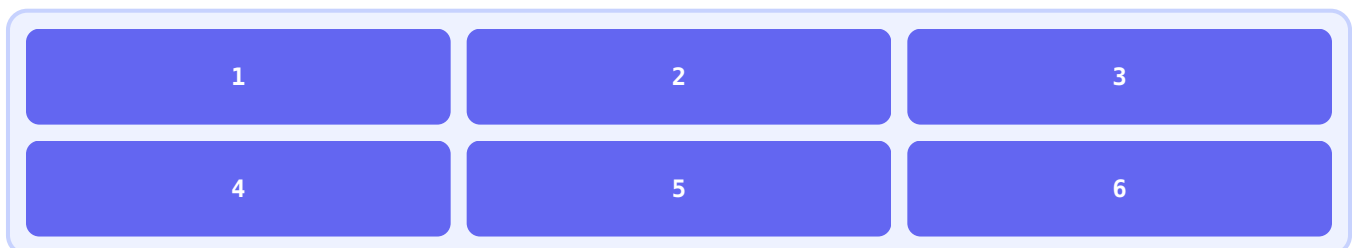
`display: flex; justify-content: center; align-items: center;` and the child sits dead centre, horizontally and vertically.

06 Grid — two-dimensional layout

Grid lays elements out in rows *and* columns at the same time. Reach for it when you have a real 2-D structure — a photo gallery, a dashboard, a card layout.

```
.gallery {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */
  gap: 12px;
}
```

CSS



Three equal columns; items flow into rows automatically.

Property	Controls
<code>grid-template-columns</code>	How many columns and their sizes. <code>fr</code> = a share of leftover space.
<code>repeat(3, 1fr)</code>	Shorthand for <code>1fr 1fr 1fr</code> .
<code>grid-template-rows</code>	Row sizes (often left automatic).
<code>gap</code>	Space between every row and column.

Use...	When you're laying out...
Flexbox	a single row or column (1-D): navbars, toolbars, button groups.
Grid	rows and columns together (2-D): galleries, page layouts, dashboards.

07 Putting it together: a styled component

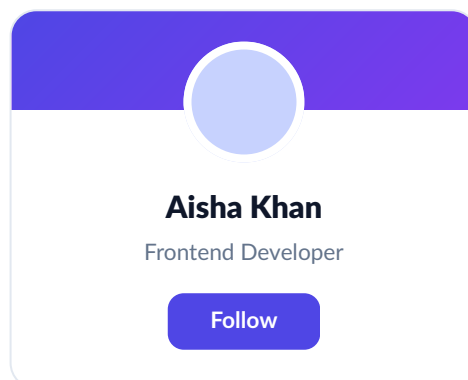
Let's combine everything — a class, the box model, colours and layout — into one reusable **profile card**.

```
<div class="card">
  <div class="banner"></div>
  <div class="avatar"></div>
  <h3 class="name">Aisha Khan</h3>
  <p class="role">Frontend Developer</p>
  <a class="btn">Follow</a>
</div>
```

HTML

```
.card {
  width: 240px;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,.12);
  text-align: center;
  overflow: hidden;
}
.avatar {
  width: 64px; height: 64px;
  border-radius: 50%;
  margin: -32px auto 0; /* pull up over banner */
}
.btn {
  display: inline-block;
  padding: 8px 24px;
  margin: 12px 0 20px;
  background: #4f46e5;
  color: white;
  border-radius: 6px;
}
```

CSS



The rendered result — built entirely from a class, the box model, and colours.

Notice every idea at work: a reusable `.card` class; **padding** and **margin** for spacing; a **border**, **border-radius** and shadow for shape; and **colours** for the accent. That is most of real-world CSS.

08 Recap & best practices

Do this	Because
Use an external stylesheet	keeps HTML clean; restyles the whole site from one file.
Style with classes , not ids	classes are reusable and easy to override.
Set <code>box-sizing: border-box</code>	widths stay predictable once you add padding.
Use padding inside, margin outside	the box model's core distinction.
Flexbox for 1-D, Grid for 2-D	each tool fits the shape of the layout.
Prefer <code>gap</code> over margins for spacing	even, predictable gaps between items.

Now build it yourself

Re-create the profile card from Section 7 in an external stylesheet, then try turning three cards into a row with Flexbox and a grid with Grid. Keep the Day 1 CSS cheat sheet beside you — muscle memory comes from typing these properties, not just reading them.